How Do We Know an ARC Solution Is Right?

Coverage, Selection, Calibration, and the N=120 Problem in ARC-AGI-2

Robert Morong · Independent research · ARC Prize 2026 Paper Track · revised July 2026

AI-assisted implementation; all calculations reproducible; every correction published regardless of direction

Abstract

ARC-AGI asks whether a system can infer a novel transformation from a few demonstrations. Two measurements sit underneath every score: whether a candidate program that fits the demonstrations will generalize, and whether a leaderboard with 120 tasks can resolve competing systems. We audit both—and correct our own first analysis in the process.

The initial experiment pooled candidate programs and reported that a demonstration-consistent program generalized at **50.0%, 86.8%, and 94.9%** after fitting one, two, and three demonstrations. Those percentages are numerically correct for this DSL's generated candidate population, but they overweight tasks that generate many programs and change both the represented task population and the held-out target as demonstration count increases. Equal-task reweighting changes the marginal rates to **45.5% [34.1, 57.0]**, **79.8% [64.7, 92.6]**, and **90.9% [72.7, 100]**, yet the trend still does not identify the effect of another demonstration.

We therefore run a pre-specified same-target experiment: for every task and each held-out demonstration, enumerate every subset of the remaining demonstrations, vary only the number fitted, and bootstrap complete task clusters. Across **1,000 training tasks and 28,476 subset cells**, fitting one demonstration gives **7.10% coverage**, **32.8% candidate reliability**, and **3.31% consensus yield**. Fitting two gives **3.83% coverage**, **50.8% candidate reliability**, and **3.03% consensus yield**. On identical tasks and held-out targets, the second demonstration reduces coverage by **3.66 percentage points [−4.63, −2.74]** and consensus yield by **0.37 points [−0.60, −0.17]**. A frozen one-shot replication on the 120 public evaluation tasks finds the same direction: coverage falls **1.24 points [−2.64, −0.23]** and consensus yield falls **0.19 points [−0.58, 0.00]**. More evidence makes the rare surviving hypotheses cleaner, but this incomplete DSL loses hypotheses faster than it gains reliability.

Selection still matters. On **224 ambiguous subset cells across 41 training tasks**, tie-aware minimum-description-length selection scores **30.0%**, versus **18.9%** for random candidate selection, an improvement of **11.1 points [4.6, 17.9]**. But the candidate oracle reaches **33.7%**, exceeding MDL by **3.65 points [0.13, 9.47]**; the earlier claim that shortest selection matched the oracle is refuted. Candidate agreement is not calibrated confidence: when all generated candidates agree, the task-weighted accuracy is only **37.8% [28.8, 47.0]** despite nominal confidence of 100%.

Finally, we turn the audit into an evidence-weighted family selector and freeze it before public-evaluation scoring. It, pure MDL, and the released vote baseline all score **0/167** outputs because the DSL generates no useful candidate on the harder evaluation tasks. The algorithmic null result locates the bottleneck: **representation and coverage, not tie-breaking**.

The leaderboard conclusion remains: at N=120, a score near 50% has an approximately ±9-point 95% interval, and small frontier gaps cannot be ranked reliably without paired per-task outcomes. We propose a reporting standard that separates coverage, conditional reliability, end-to-end yield, selection policy, calibration, cost, and task-cluster uncertainty. The contribution is not another leaderboard point. It is an audit of the ruler—and a public example of revising a strong result when a better experiment overturns its interpretation.

1. Introduction

The Abstraction and Reasoning Corpus (ARC-AGI) is designed to test skill acquisition rather than retrieval: a solver sees a handful of input-output demonstrations and must infer the transformation for one or more unseen grids [1, 2]. ARC-AGI-2 raises the difficulty with larger grids, interacting rules, symbolic reinterpretation, and an explicit efficiency constraint.

Almost all public attention goes to one number: the fraction of evaluation tasks solved. This paper examines the measurement acts beneath that number.

The first is **hypothesis acceptance**. A symbolic search system, an LLM-generated program, or a test-time-trained model proposes hypotheses and checks them against the demonstrations. A candidate that reproduces every demonstration is called consistent. But consistency is not the endpoint; generalization to the unseen target is. With few examples and a misspecified hypothesis class, the two can diverge.

The second is **hypothesis selection**. Several programs may fit the same demonstrations while predicting different test grids. The solver must choose among them. Voting, first-found order, minimum description length (MDL), learned priors, and ensembles are not implementation details; they can alter the benchmark score.

The third is **system ranking**. ARC-AGI-2's public evaluation set contains 120 tasks. At that sample size, small score differences carry substantial sampling uncertainty, even before accounting for repeated public evaluation, cost differences, or shared task errors.

Our first paper draft correctly identified underdetermination but made two important statistical mistakes. It pooled candidate programs rather than tasks, and its prefix ablation changed the held-out target as the number of fitted demonstrations increased. We keep the original results in the repository, build the corrected experiment around those criticisms, and revise every claim against the new evidence.

Contributions

1. **A correction of program-weighted calibration.** We report both candidate-population and equal-task estimands with task-cluster uncertainty.
2. **A same-target all-subsets experiment.** Demonstration count varies while task and held-out target remain fixed.
3. **A three-part measurement decomposition.** Coverage, conditional reliability, and end-to-end yield are reported separately.
4. **A tie-aware selection audit.** Random selection, enumeration-order shortest, tie-aware MDL, consensus, and a candidate oracle are compared on identical ambiguous cells.
5. **A confidence-calibration audit.** Candidate agreement is tested as a probability forecast rather than described qualitatively.
6. **A frozen algorithmic extension.** Program-family reliability is learned from training demonstrations and evaluated once on public evaluation outputs; its null result is retained.
7. **A leaderboard uncertainty standard.** Confidence intervals, paired comparisons, cost, and data-use status become required parts of a capability claim.

---

2. Prior Work

ARC solving. Major approaches include domain-specific-language search, neural transduction, LLM-guided program generation, test-time training, and hybrids [3–6]. Inductive and transductive systems often solve different task subsets [3]. This complementarity makes candidate generation and selection central rather than peripheral.

Calibration and selective prediction. Calibration asks whether stated confidence matches empirical frequency [7]. In ARC, a solver can derive confidence from candidate counts, vote concentration, model likelihood, self-consistency, or learned verifier scores. None is calibrated merely because it is monotone.

Minimum description length. MDL supplies a prior favoring simpler hypotheses. In program synthesis its effectiveness depends on the representation language: a transformation is short only relative to a chosen primitive set. Equal-complexity ties must also be resolved explicitly.

Benchmark uncertainty. A benchmark score is a sample proportion over tasks. Wilson intervals describe uncertainty in a single score [8]; paired outcomes permit McNemar or exact binomial comparisons between systems [9]. Public leaderboard reuse adds an adaptive-selection problem beyond ordinary binomial uncertainty.

---

3. Data and Solver

3.1 Data

We use the official public ARC-AGI-2 corpus: **1,000 training tasks and 120 evaluation tasks**. Ground-truth outputs are used only where the official public release provides them. The same-target methodology uses demonstration pairs; the frozen solver benchmark uses the public evaluation outputs as a one-shot holdout after the rules were registered.

The official data commit used by the workflows is recorded in `results/arc_agi_2_data_commit.txt`. No private test labels or private-leaderboard feedback enter development.

3.2 Diagnostic DSL

The deterministic CPU-only solver contains geometric transforms, cropping, connected-component and object operations, tiling, logical half-plane operations, color maps, and depth-two compositions. Some operation parameters are inferred from the demonstrations. A candidate is accepted only if it reproduces every fitted output exactly.

The DSL is intentionally incomplete. Its purpose is to expose acceptance and selection behavior, not to claim frontier capability. That incompleteness becomes a central empirical finding.

3.3 Why the original prefix experiment was insufficient

For demonstrations d0…dD−1, the original analysis fitted d0…d{k−1} and tested dk. Thus:

- k=1 and k=2 tested different outputs;
- only tasks with enough demonstrations contributed at larger k;
- tasks that generated more candidates carried more weight.

The design is a valid description of generated candidate programs along a particular prefix path. It is not an identified effect of adding evidence.

---

4. Corrected Methodology

4.1 Equal-task legacy reweighting

Within each `(task, k)` cell we compute the fraction of consistent programs that generalize. We then average those rates equally over represented tasks. Uncertainty comes from nonparametric resampling of complete task clusters.

This corrects candidate-count weighting, but not changing targets or changing task composition; its rates remain descriptive.

4.2 Same-target all-subsets cross-fold design

For every task and each demonstration selected as holdout h:

1. remove h from the fitted set;
2. enumerate every subset of size k from the remaining demonstrations;
3. build the candidate DSL from that subset;
4. retain exact-consistent executable programs;
5. evaluate them on the fixed held-out input h;
6. repeat for every feasible k.

Subset choices are averaged within `(task, holdout, k)`, holdouts are averaged within task, and tasks are weighted equally. Bootstrap resampling occurs only at the task level. The training run contains **28,476 subset cells** and **8,092 task/holdout/k folds**.

4.3 Three distinct quantities

For a selection rule S:

- **Coverage:** probability that at least one executable consistent candidate exists.
- **Conditional reliability:** correctness among generated candidates or covered cells.
- **End-to-end yield:** probability S returns the held-out output, counting uncovered cells as failures.

A system can improve conditional reliability while reducing yield if additional demonstrations eliminate too many candidates from a misspecified hypothesis class.

4.4 Selection rules

We evaluate:

- **Random candidate:** expected correctness of a uniformly selected surviving program.
- **Legacy shortest:** the first minimum-complexity program in DSL enumeration order.
- **MDL-random:** random choice among all minimum-complexity candidates.
- **MDL-vote:** vote among minimum-complexity candidate outputs.
- **Consensus:** vote across all candidates, with deterministic complexity tie-breaking.
- **Candidate oracle:** succeeds if any generated candidate predicts the holdout.

The distinction between legacy shortest and tie-aware MDL prevents arbitrary enumeration order from masquerading as Occam's razor.

4.5 Confidence audit

The modal candidate-vote fraction is treated as a forecast probability. We report reliability bins, task-weighted Brier score, and confidence-minus-accuracy gaps.

4.6 Registered one-shot replication

The design and interpretation rules were frozen in `HYPOTHESIS-crossfold-v2.md` before the full public-evaluation run. The 120 evaluation tasks are then analyzed once with the same code. Evaluation results do not alter the v2 method.

4.7 Evidence-weighted selector

A separate pre-registration freezes an algorithmic extension:

1. learn program-family reliability from training-task cross-validation;
2. shrink sparse family rates toward the task-weighted global mean;
3. update them with same-task leave-one-demonstration-out evidence;
4. deduplicate syntactic family votes;
5. penalize description length;
6. return two distinct outputs.

The algorithm is compared with released consensus and pure MDL on the public evaluation outputs. Promotion requires a paired improvement; ties and losses are reported.

4.8 Leaderboard statistics

For a score $\hat{p}$ over N tasks we report 95% Wilson intervals. System comparisons should use paired per-task outcomes; where unavailable, an unpaired two-proportion calculation is only a conservative approximation. Power calculations use $\alpha=0.05$ and 80% power.

---

5. Results

5.1 The original curve survives reweighting—but not causal identification

Demonstrations fit	Candidate-program weighted	Equal-task weighted	95% task-cluster CI	Tasks
1	50.0%	**45.5%**	[34.1, 57.0]	67
2	86.8%	**79.8%**	[64.7, 92.6]	31
3	94.9%	**90.9%**	[72.7, 100]	11

Candidate-rich tasks modestly inflated the original figures. More importantly, the represented tasks and held-out outputs change with k. On the 31 tasks shared between k=1 and k=2 in the legacy prefix design, the task-weighted difference is **−4.7 percentage points**, not +34 points, and its interval includes zero. The dramatic marginal rise was primarily composition and target selection, not an identified evidence-count effect.

5.2 Same-target training result: reliability rises while coverage collapses

k fitted	Tasks	Coverage	Candidate reliability	Consensus yield	Oracle yield
1	1,000	**7.10%** [5.71, 8.58]	**32.8%** [25.1, 40.4]	**3.31%** [2.31, 4.40]	**3.44%** [2.41, 4.55]
2	842	**3.83%** [2.66, 5.08]	**50.8%** [37.8, 64.0]	**3.03%** [1.94, 4.22]	**3.03%** [1.94, 4.22]
3	267	**4.37%** [2.14, 6.90]	**63.4%**	**3.84%**	**3.84%**

The conditional candidate population looks cleaner at larger k, but the solver is able to express far fewer hypotheses. On the identical 842 tasks and 2,916 held-out targets contributing to both k=1 and k=2:

Same-target change, k=2 minus k=1	Difference	95% task-cluster CI
Coverage	**−3.66 pp**	[−4.63, −2.74]
Random-selection yield	**−0.25 pp**	[−0.50, −0.00]
MDL-vote yield	**−0.42 pp**	[−0.66, −0.20]
Consensus yield	**−0.37 pp**	[−0.60, −0.17]
Oracle yield	**−0.46 pp**	[−0.70, −0.25]

Against the registered rule, the primary consensus-yield effect is **negative**, not positive. More demonstrations do not harm ARC reasoning in general. They expose that this DSL is representation-limited: added constraints remove its approximate hypotheses faster than they identify a correct one.

5.3 One-shot evaluation replication

The harder public evaluation tasks sharply reduce coverage.

k fitted	Coverage	Consensus yield	Candidate reliability
1	**1.03%** [0.17, 2.25]	**0.139%** [0, 0.417]	12.5% [0, 50.0]
2	**0.194%** [0, 0.581]	**0%**	0%

For the same held-out target, k=2 minus k=1 gives:

- coverage: **−1.24 pp [−2.64, −0.23]**;
- consensus yield: **−0.194 pp [−0.581, 0.000]**;
- oracle yield: **−0.194 pp [−0.581, 0.000]**.

Every pre-specified primary effect has the same negative direction in training and evaluation. The magnitude is small because evaluation yield is nearly zero, but the representation bottleneck replicates.

Evaluation coverage at k=1 is **6.07 points below training** [−7.84, −4.26], and consensus yield is **3.17 points below training** [−4.28, −2.14]. This is direct evidence of a distribution and difficulty shift relevant to public-to-private leaderboard expectations.

5.4 MDL helps, but it is not an oracle

Across **224 ambiguous subset cells from 41 training tasks**:

Selection rule	Task-weighted accuracy	95% CI
Random candidate	**18.9%**	[10.8, 27.8]
Legacy first-shortest	**31.2%**	[18.5, 44.8]
Random minimum-complexity tie	**30.4%**	[18.3, 43.5]
Tie-aware MDL vote	**30.0%**	[17.7, 43.1]
All-candidate consensus	**27.4%**	[15.6, 40.0]
Candidate oracle	**33.7%**	[20.5, 47.6]

Tie-aware MDL improves over random selection by **11.1 pp [4.6, 17.9]**. Consensus improves over random by **8.5 pp [2.6, 14.8]**. The useful part of the original Occam result survives.

The exact oracle claim does not. Oracle exceeds MDL vote by **3.65 pp [0.13, 9.47]**. Therefore “shortest is correct whenever any candidate is correct” is withdrawn. Description length is a strong prior in this DSL, not a complete selection solution.

5.5 Candidate agreement is severely overconfident

The original draft described candidate agreement as calibrated confidence. The expanded cross-fold audit reverses that conclusion.

- Task-weighted Brier score: **0.542 [0.465, 0.618]**.
- Mean absolute confidence-error gap: **59.5 points [52.1, 66.7]**.
- When modal vote fraction is exactly **1.0**, task-weighted accuracy is only **37.8% [28.8, 47.0]**.

Unanimity inside a misspecified candidate class is not evidence of truth. It can mean only that every available program shares the same representational blind spot.

5.6 Evidence-weighted selection cannot rescue absent representations

Program-family priors were learned from all 1,000 training tasks and frozen before evaluation. On 167 public-evaluation test outputs:

Method	pass@1	pass@2
Released vote + MDL baseline	0/167	0/167
Pure MDL	0/167	0/167
Evidence-weighted family selector	0/167	0/167

All paired tests are null because no method solves an output. The submission metadata shows the operative failure: the evaluation tasks almost never produce a useful candidate, and the full-test solver returns fallbacks. Selection improves decisions only after representation supplies viable alternatives.

This negative result narrows the contest roadmap. The next gain must come from richer induction or transduction—object-centric abstractions, learned program proposals, test-time training, or a hybrid—not another tie-breaker over the same DSL.

5.7 The N=120 leaderboard remains statistically coarse

At p≈0.50 and N=120, a 95% Wilson interval is approximately ±9 percentage points. An unpaired calculation requires roughly:

- **1,565 tasks** to detect a five-point gap at 80% power;
- **4,356 tasks** for a three-point gap;
- **9,800 tasks** for a two-point gap.

Paired tests can be more powerful when systems fail on different tasks, which is precisely why per-task outcomes should accompany ranking claims. A decimal leaderboard gap without a paired test is not a demonstrated capability difference.

Evidence Figures

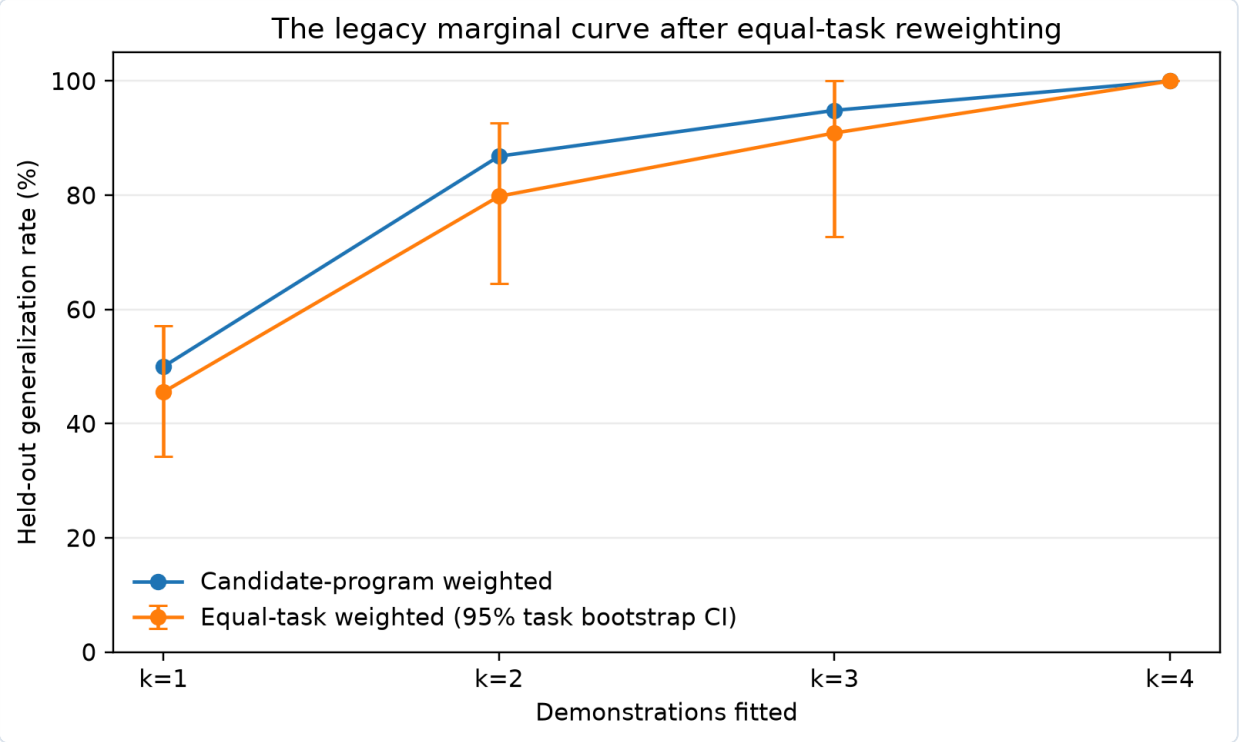

Figure 1. The original candidate-program-weighted curve and the corrected equal-task estimates. The marginal rise remains descriptive because task composition and held-out targets change with k.

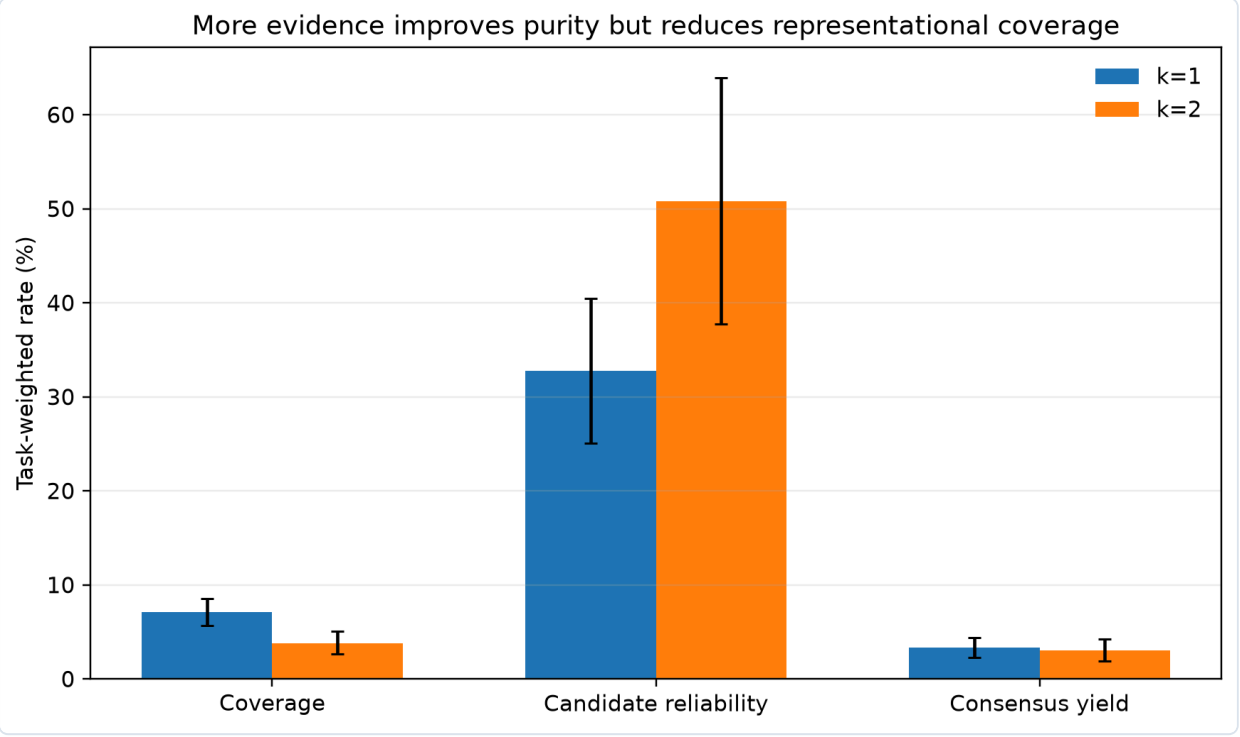

Figure 2. Added demonstrations increase conditional candidate reliability while reducing the diagnostic DSL's coverage and leaving end-to-end consensus yield near three percent.

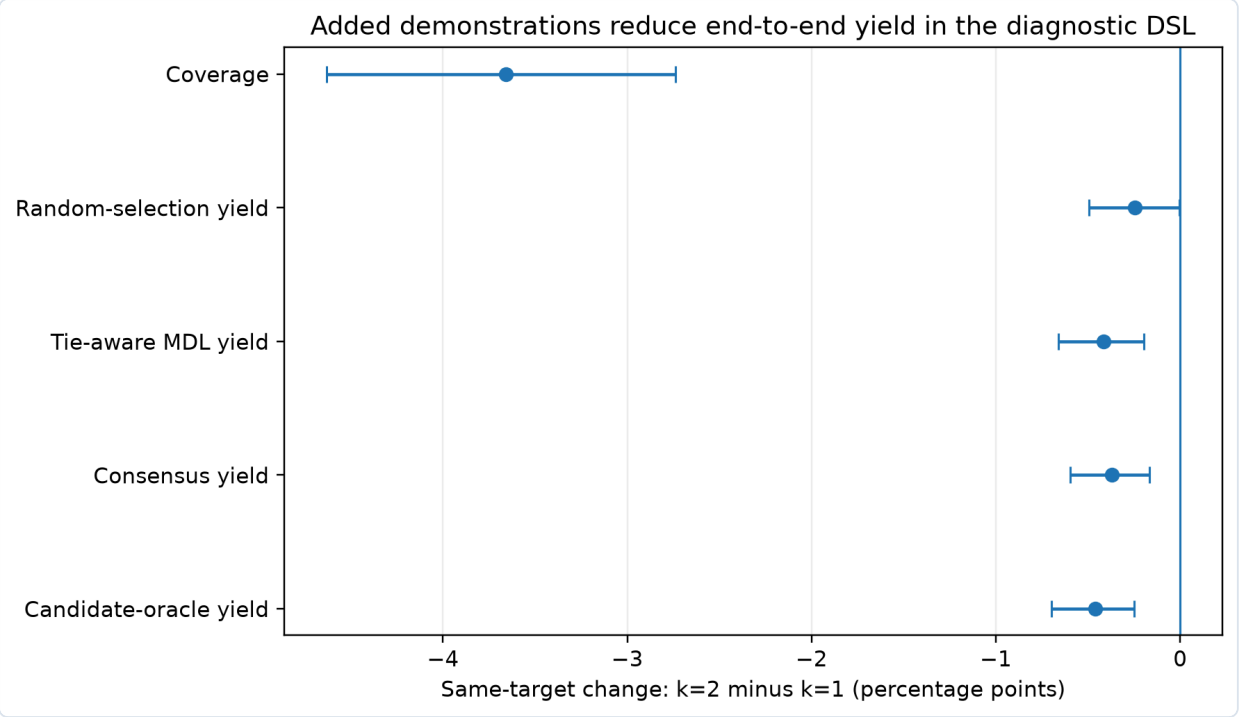

Figure 3. Same-task, same-target effects of fitting two rather than one demonstration. Every end-to-end quantity moves downward in this representation-limited DSL.

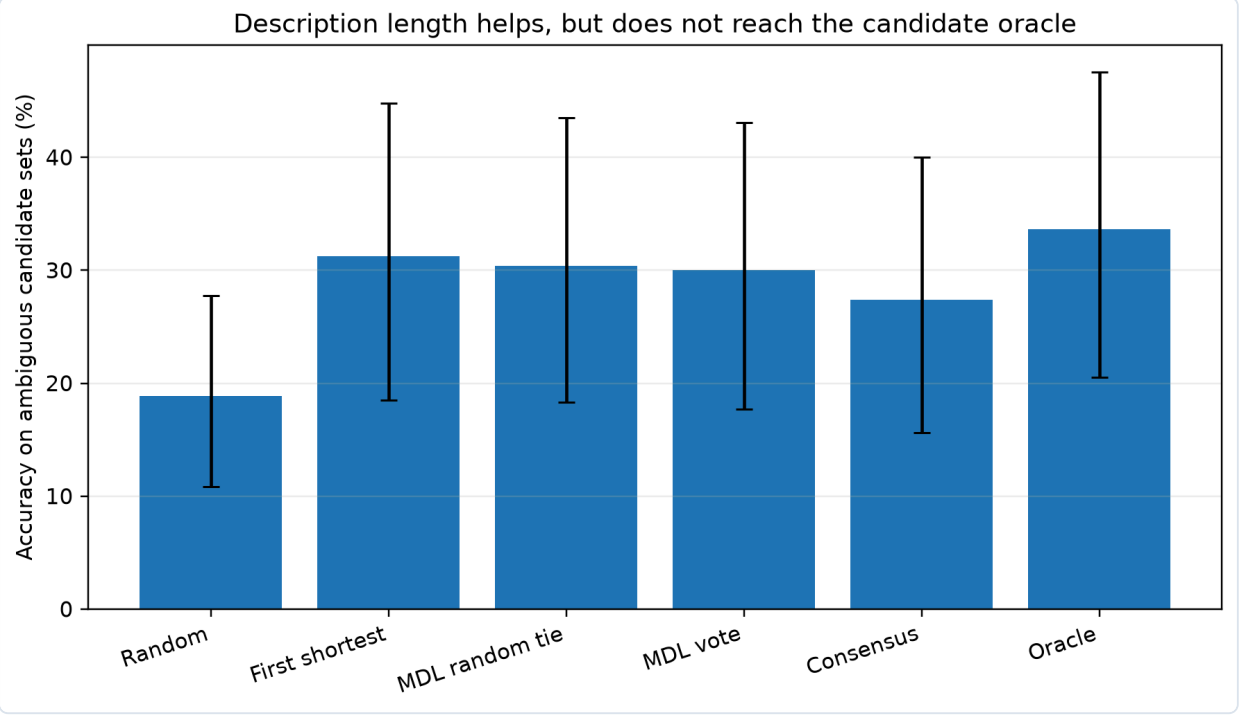

Figure 4. Selection on ambiguous candidate sets. Description length improves over random selection but does not attain the candidate-set oracle.

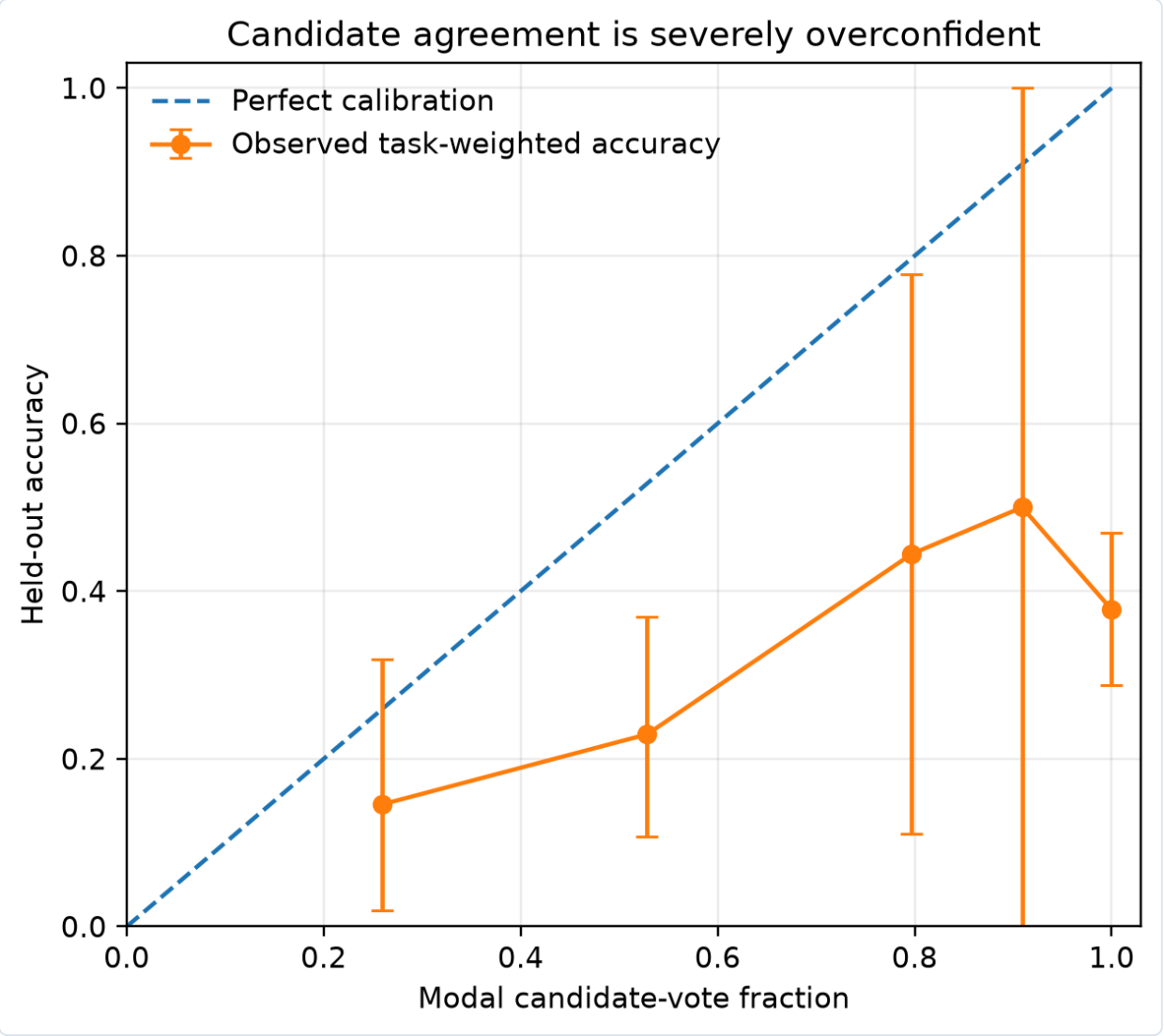

Figure 5. Modal candidate agreement is severely overconfident. Unanimity within a restricted grammar does not imply correctness.

6. Discussion

6.1 Evidence can improve purity while reducing utility

The central corrected result is not that demonstrations are harmful. It is that the effect of evidence depends on the interaction between evidence and hypothesis class.

For this DSL, a second demonstration increases the purity of the rare candidates that survive, but it also eliminates many tasks for which the DSL previously produced an approximate hypothesis. Because coverage falls more than reliability rises, end-to-end yield declines. This is analogous to a selective classifier raising precision by abstaining on nearly everything.

ARC papers should therefore avoid reporting conditional accuracy alone. At minimum they should publish:

1. candidate or answer coverage;
2. accuracy conditional on coverage;
3. unconditional yield;
4. abstention policy;

5. cost.

6.2 Occam is useful, not magical

MDL's positive effect survives a broader, task-weighted audit. The exact oracle equality does not. Both findings matter.

The result supports description length as a default prior when candidates disagree, while warning that its performance is language-dependent and tie-dependent. A richer DSL changes which hypothesis is shortest; an arbitrary enumeration order is not a scientific selection rule.

6.3 Consensus is not confidence

Candidate agreement is especially dangerous when every candidate comes from the same restricted grammar. Correlated errors create unanimous but wrong predictions. A solver that exposes modal vote fraction as confidence would be dramatically overconfident here.

ARC systems should publish empirical calibration curves for whatever confidence mechanism they use and report selective-risk curves when abstention is allowed.

6.4 The public evaluation shift is measurable

The diagnostic DSL covers 7.1% of training task/target situations at k=1 but only 1.0% on evaluation. The one-shot drop is not a leaderboard artifact; it appears in demonstration-pair cross-validation before test scoring. This offers a reproducible way to quantify benchmark shift without repeated leaderboard probing.

6.5 What the null solver result tells the contest effort

The evidence-weighted selector does not improve a zero-coverage representation. That is a useful stop signal. Further work should prioritize candidate generation:

- object and relation discovery;
- reusable subprogram induction;
- learned proposal models;
- test-time adaptation;
- induction-transduction ensembles;
- calibrated abstention and compute allocation.

The measurement audit becomes a diagnostic harness for those systems: every new generator can be decomposed into coverage, reliability, yield, ambiguity, selection gain, and calibration.

---

7. Proposed ARC Reporting Standard

Every ARC-AGI capability report should include:

1. **Data status.** Training, public evaluation, semi-private, or private; number of adaptive submissions.
2. **Task count and intervals.** Wilson or exact intervals on every aggregate score.
3. **Paired outcomes.** McNemar or exact discordant-pair tests for system comparisons.
4. **Coverage.** Fraction of tasks/outputs for which the system produces a valid answer without fallback.
5. **Conditional reliability and unconditional yield.** Report both.
6. **Selection rule.** Voting, MDL, verifier, first-found, or ensemble logic, including tie handling.
7. **Confidence calibration.** Reliability diagrams, Brier/ECE, and selective risk.

8. **Cost.** Compute, wall time, model calls, and pass@k.
9. **Representation ablations.** Separate candidate-generation improvements from selection improvements.
10. **Reproducibility.** Code, fixed seeds, pinned data commit, raw per-task outcomes, and an executable environment.

---

8. Limitations

The diagnostic DSL covers only a small portion of ARC-AGI-2, especially evaluation. Its conclusions apply directly to demonstration-consistent programs produced by this grammar; they do not establish that every neural or LLM solver will show the same coverage-reliability tradeoff.

Although the evaluation analysis was registered and frozen, the public evaluation set is public. It is a stronger replication than reusing training tasks but weaker than the private competition test set. The evidence-weighted selector's zero score should not be generalized to richer systems.

The training ambiguous-selection sample contains 41 tasks. Intervals remain wide, especially at larger k. MDL complexity is specific to this DSL. Confidence calibration is measured over correlated candidate families generated by one solver, not over the universe of possible hypotheses.

Leaderboard power calculations use simplified Bernoulli assumptions. Paired per-task data, heterogeneous task difficulty, and adaptive leaderboard use require richer models; the simple calculation is intended to show scale, not provide a complete benchmark-design theory.

---

9. Conclusion

The first version of this study found an appealing story: one demonstration was a coin flip, three demonstrations were reliable, shortest programs matched an oracle, and candidate agreement provided confidence. The better experiment changes that story.

After equal-task weighting and fixed-target control, additional demonstrations do not improve this solver's end-to-end yield. They increase the reliability of the rare hypotheses that remain while sharply reducing coverage. MDL still improves selection, but it does not reach the oracle. Candidate unanimity is severely overconfident. An evidence-weighted selector cannot overcome a hypothesis class that fails to represent the evaluation tasks.

These are not failures of the research program. They are its result. ARC-AGI is designed to test generalization under sparse evidence; its own progress claims should be held to the same standard. The field needs richer solvers, but it also needs an honest ruler: task-weighted uncertainty, same-target controls, explicit coverage, calibrated confidence, paired comparisons, and public correction when a stronger test overturns a cleaner narrative.

---

10. Reproducibility

The repository releases:

- `task_clustered_analysis.py` — equal-task correction of the legacy prefix analysis;
- `crossfold_ablation.py` — complete same-target, all-subsets experiment;
- `crossfold_analysis.py` — task-cluster bootstrap, selection, and calibration audit;
- `crossfold_replication.py` — frozen training-to-evaluation comparison;
- `evidence_weighted_solver.py` — family-prior selector and two-attempt submission builder;

- `benchmark_solver_v2.py` — paired frozen public-evaluation benchmark;
- `HYPOTHESIS-crossfold-v2.md` and `HYPOTHESIS-evidence-weighted-solver.md` — pre-specified interpretation rules;
- `results/` — machine-readable summaries, task tables, predictions, and submission artifacts;
- `site/` — live evidence dashboard.

The complete analysis is CPU-only. Workflows pin the official ARC-AGI-2 data commit, preserve raw artifacts, and publish negative results.

References

[1] F. Chollet. “On the Measure of Intelligence.” arXiv:1911.01547, 2019.
[2] F. Chollet, M. Knoop, G. Kamradt, B. Landers, H. Pinkard. “ARC-AGI-2: A New Challenge for Frontier AI Reasoning Systems.” arXiv:2505.11831, 2025.
[3] W.-D. Li, K. Hu, C. Larsen, et al. “Combining Induction and Transduction for Abstract Reasoning.” arXiv:2411.02272, 2024.
[4] E. Akyürek, M. Damani, A. Zweiger, et al. “The Surprising Effectiveness of Test-Time Training for Abstract Reasoning.” arXiv:2411.07279, 2024.
[5] A. Jolicoeur-Martineau. “Less is More: Recursive Reasoning with Tiny Networks.” arXiv:2510.04871, 2025.
[6] I. Liao, A. Gu. “ARC-AGI Without Pretraining.” arXiv:2512.06104, 2025.
[7] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger. “On Calibration of Modern Neural Networks.” ICML, 2017.
[8] E. B. Wilson. “Probable Inference, the Law of Succession, and Statistical Inference.” JASA 22(158), 1927.
[9] Q. McNemar. “Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages.” Psychometrika 12(2), 1947.